

GEN-KNOB TUNER

Internal Supervisor: Dr. Buddhika Karunaratne

External Supervisor: Dr. Srinath Perera

Team Members:

Madara Mendis

Madushan Suriyanbandara

Nisith Dissanayake

Nithika Dias

INTRODUCTION

Knob tuning is process of configuring database system's internal parameters (knobs) to optimize query execution.

- Objectives:
 - Efficient resource utilization
 - Minimal latency
 - Maximal throughput for varying workloads.

SOME DEFINITIONS TO REMEMBER...

What is a Knob Configuration?

A set of parameters which can change the database internal settings

```
"max_wal_senders": 39.5,  
"autovacuum_max_workers": 151.5,  
"max_connections": 1138.1666666666665,  
"wal_buffers": 46216.944444444444,  
"shared_buffers": 320031.27777777777,  
"autovacuum_analyze_scale_factor": 52.0,  
"autovacuum_analyze_threshold": 710745667.5555553,  
"autovacuum_naptime": 853838.94444444443,  
"autovacuum_vacuum_cost_delay": 74.27777777777771,  
"autovacuum_vacuum_cost_limit": 5740.055555555555,  
"autovacuum_vacuum_scale_factor": 54.0,  
"autovacuum_vacuum_threshold": 787531196.4444444,  
"backend_flush_after": 132.44444444444434,  
"bgwriter_delay": 9232.666666666668,  
"bgwriter_flush_after": 237.0,  
"bgwriter_lru_maxpages": 929.3888888888889,  
"bgwriter_lru_multiplier": 7.5,  
"checkpoint_completion_target": 0.5,  
"checkpoint_flush_after": 238.88888888888886,  
"checkpoint_timeout": 3337.5,  
"commit_delay": 36205.111111111095,  
"commit_siblings": 359.6111111111111,
```

What is a Workload?

A set of SQL queries inside one file

```
E2ETune-AI4DB > inference_test >  job_0-3_first_3.sql
```

```
1  SELECT t.title, COUNT(c.movie_id) AS num_cast_members FROM title AS t JOIN cast_info AS c ON t.id = c.movie_id WHERE
2  SELECT k.keyword, COUNT(mk.movie_id) AS num_movies FROM keyword AS k JOIN movie_keyword AS mk ON k.id = mk.keyword_id
3  SELECT n.name, COUNT(ci.person_id) AS num_roles FROM name AS n JOIN cast_info AS ci ON n.id = ci.person_id WHERE ci.r
4  SELECT cn.name, cn.country_code, COUNT(mc.movie_id) AS num_movies FROM company_name AS cn JOIN movie_companies AS mc
5  SELECT n.name, COUNT(ci.movie_id) AS num_movies, COUNT(DISTINCT ci.person_id) AS num_actors FROM name AS n JOIN cast_
6  SELECT COUNT(*) FROM movie_info WHERE info_type_id = (SELECT id FROM info_type WHERE info = 'rating') AND CAST(info AS
7  SELECT title.title, AVG(CAST(movie_info.info AS FLOAT)) as average_rating FROM title JOIN movie_info ON title.id = mo
8  SELECT title.title, COUNT(movie_keyword.keyword_id) as keyword_count FROM title JOIN movie_keyword ON title.id = movi
9  SELECT t.title, COUNT(c.id) AS cast_count FROM title t JOIN cast_info c ON t.id = c.movie_id WHERE t.kind_id = 3 GROU
10 SELECT k.keyword, COUNT(mk.movie_id) AS movie_count FROM keyword k JOIN movie_keyword mk ON k.id = mk.keyword_id GROU
11 SELECT n.name, COUNT(ci.id) AS role_count FROM name n JOIN cast_info ci ON n.id = ci.person_id WHERE ci.role_id = 1 G
12
```

What are Query Plans, Workload Features, and Internal Metrics?

Query plans describe how the database will retrieve and combine rows to produce the query result.

Workload features are the specific attributes of the workload, like ratio of read/write, average number of joins etc.

Internal Metrics are the collected metrics from postgres when the workload has run. These include the number of tuples fetched, blocks read etc.

PROBLEM STATEMENT

Traditional database knob tuning, primarily reliant on manual efforts by DBAs, faces significant challenges that lead to suboptimal performance and increased operational costs. These issues are caused by the sheer number of configuration parameters, their intricate interdependencies, and the constant evolution of workloads and hardware environments.

MOTIVATION

- So there's need for a fully local, automated tuning engine that:
 - Eliminates manual experimentation
 - Preserves sensitive performance metrics
 - Dynamically adapts to different database types & versions
 - Operates without retraining



Motivation: Build a secure, intelligent, version-agnostic automated knob tuning system

RESEARCH OBJECTIVES

- Automate the manual trial-and-error suboptimal knob tuning process in relational databases.
- Engineer and develop a fully local tuning engine to optimize databases.
- Enable the system to tune any database version or type by dynamically retrieving technical knowledge without retraining.

LITERATURE REVIEW

Heuristic Knob Tuning

Bayesian Optimization
based Knob Tuning

Reinforcement Learning
based Knob Tuning

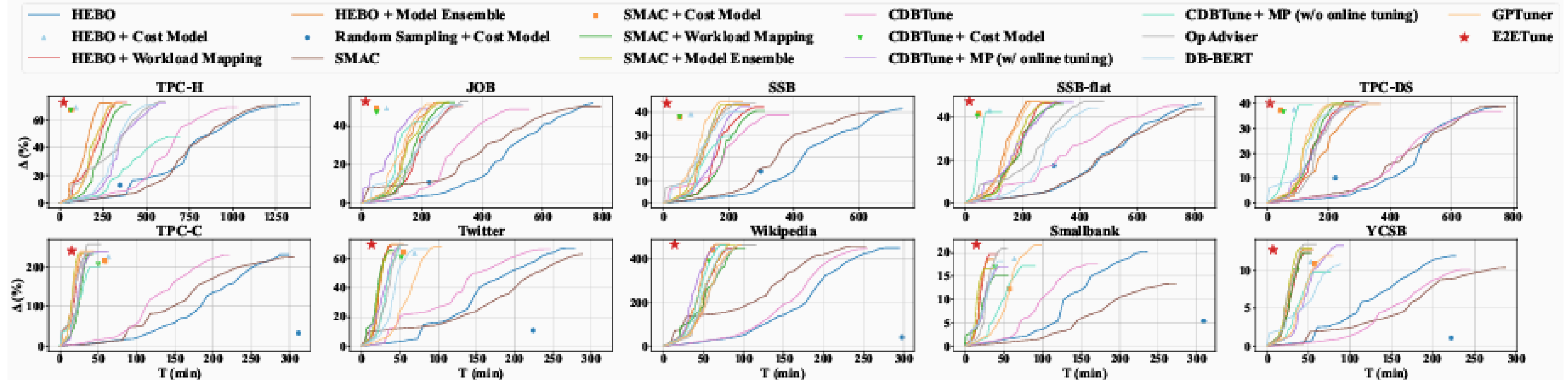
LLM based Knob
Tuning

PGTune(2011)
OpenTuner(2014)
BestConfig(2017)

SMAC(2011)
HEBO(2020)
CGPTuner(2021)
GPtuner(2023)

CDBTune(2019)
DB-BERT(2022)
OpAdviser (2023)

E2ETUNE(2024)



(a) In-Schema Evaluation

FURTHER ANALYSIS :

PROBLEMS WITH E2ETUNE

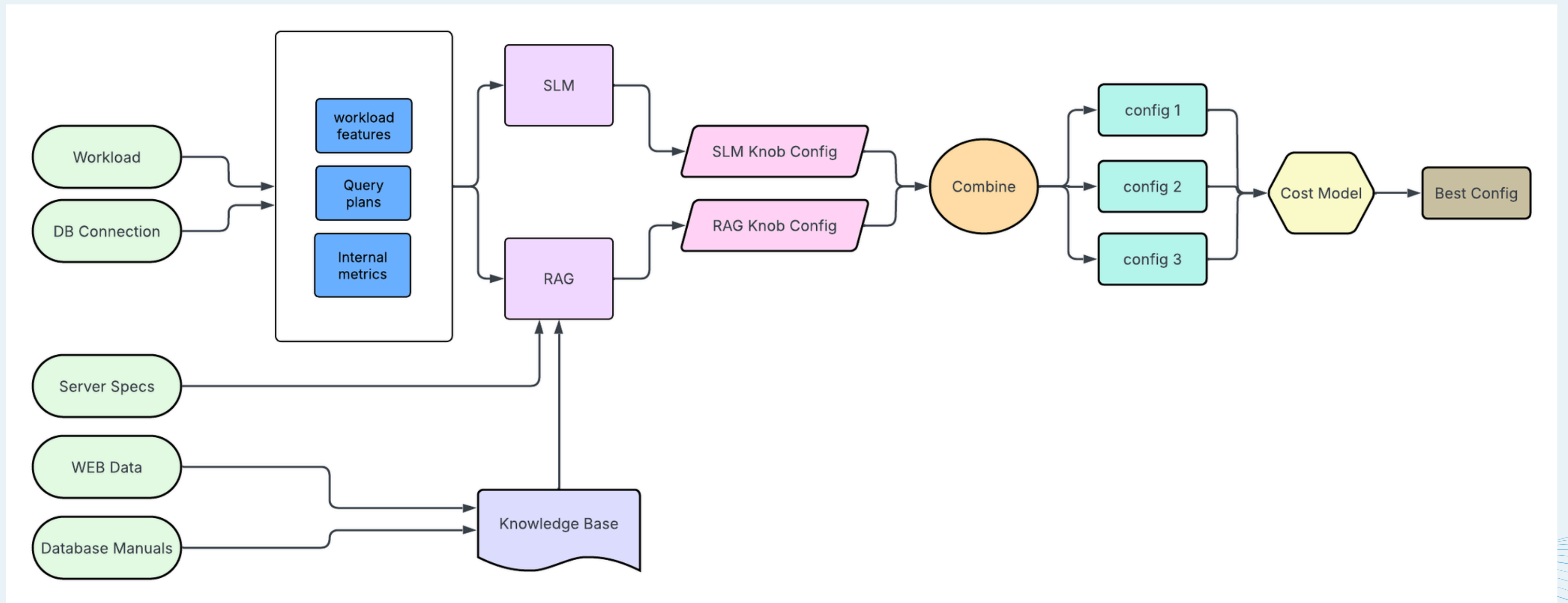
- Model is available on Huggingface hub but not a complete tool to get config directly or to update knobs automatically.
- Only works with postgresSQL.
- Takes around 30mins to generate one configuration on CPU.

```
Model Output:
[INST] workload features: size of workload: 15.0; read ratio: 1.0; group by ratio: 0.8; order by ratio:
0.87; avg query length: 56.2; number of joins: 2; filter ratio: 0.5;
query plans in workload: Aggregate(cost=1000.0)(Seq Scan(cost=650.0)); Nested Loop(cost=2000.0)(Index Sc
an(cost=1200.0); Seq Scan(cost=800.0));
inner metrics: buffer hit ratio: 0.99; average response time: 120.4ms; lock wait: 0.02; rows returned: 1
023; deadlocks: 0; [/INST] {"max_wal_senders": "10% to 20%", "autovacuum_max_workers": "40% to 50%", "ma
x_connections": "50% to 60%", "wal_buffers": "80% to 90%", "shared_buffers": "00% to 10%", "autovacuum_a
nalyze_scale_factor": "40% to 50%", "autovacuum_analyze_threshold": "70% to 80%", "autovacuum_naptime":
"40% to 50%", "autovacuum_vacuum_cost_delay": "90% to 100%", "autovacuum_vacuum_cost_limit": "60% to 7
0%", "autovacuum_vacuum_scale_factor": "20% to 30%", "autovacuum_vacuum_threshold": "70% to 80%", "backe
nd_flush_after": "30% to 40%", "bgwriter_delay": "90% to 100%", "bgwriter_flush_after": "20% to 30%", "b
gwriter_lru_maxpages": "70% to 80%", "bgwriter_lru_multiplier": "10% to 20%", "checkpoint_completion_tar
get": "90% to 100%", "checkpoint_flush_after": "70% to 80%", "checkpoint_timeout": "60% to 70%", "commit
_delay": "90% to 100%", "commit_siblings": "90% to 100%", "cursor_tuple_fraction": "00% to 10%", "deadlo
ck_timeout": "70% to 80%", "default_statistics_target": "80% to 90%", "effective_cache_size": "60% to 7
0%", "effective_io_concurrency": "10% to 20%", "from_collapse_limit": "40% to 50%", "geqo_effort": "70%
to 80%", "geqo_generations": "70% to 80%", "geqo_pool_size": "90% to 100%", "geqo_seed": "90% to 100%",
"geqo_threshold": "60% to 70%", "join_collapse_limit": "50% to 60%", "maintenance_work_mem": "10% to 2
0%", "temp_buffers": "10% to 20%", "temp_file_limit": "middle", "vacuum_cost_delay": "40% to 50%", "vacu
um_cost_limit": "80% to 90%", "vacuum_cost_page_dirty": "30% to 40%", "vacuum_cost_page_hit": "50% to 6
0%", "vacuum_cost_page_miss": "90% to 100%", "wal_writer_delay": "60% to 70%", "work_mem": "60% to 70%"}
```

PROPOSED METHOD

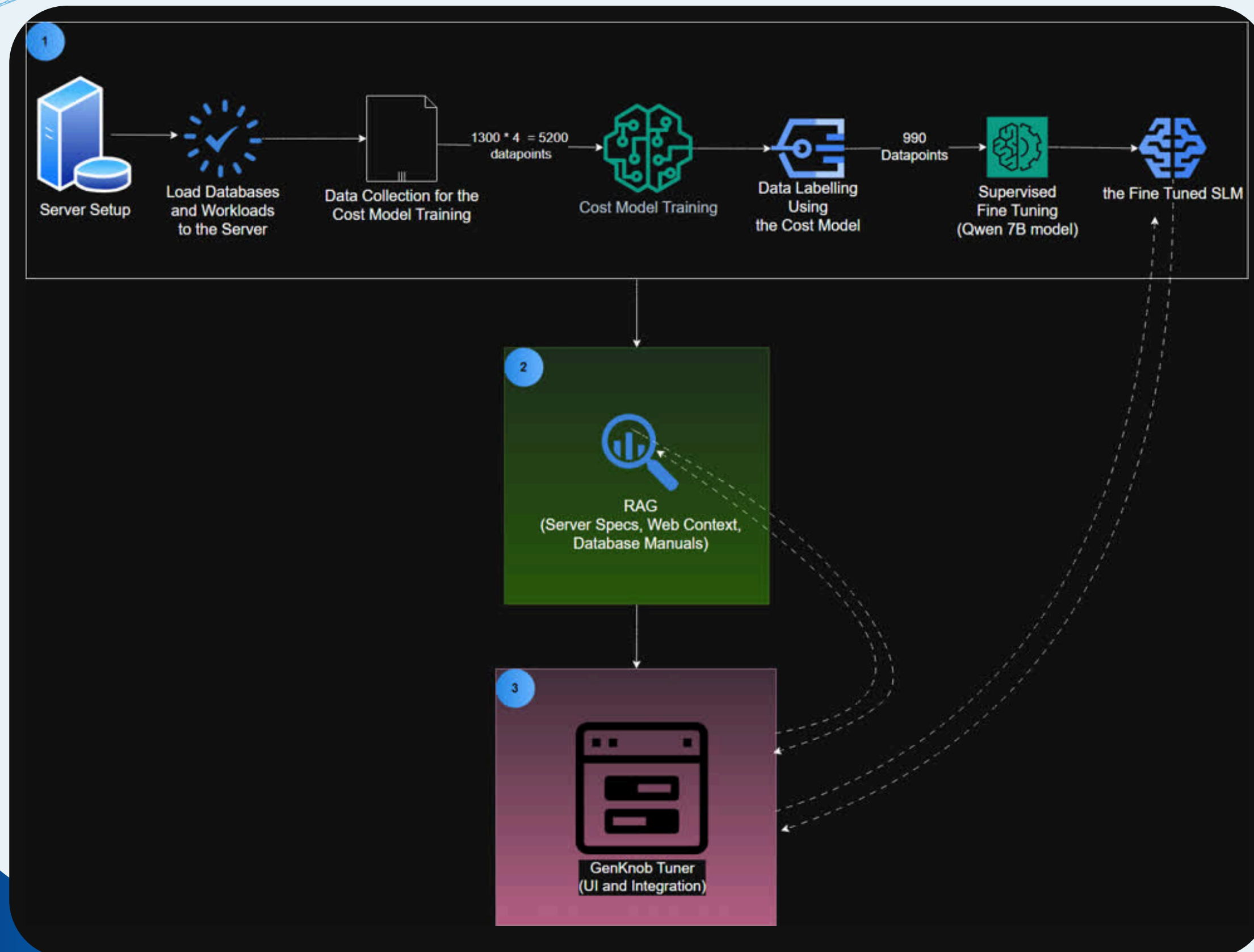
- We propose **GEN-KNOB Tuner**, an autonomous, edge-deployed database optimization engine designed for privacy-preserving, local inference.
- The system leverages a **lightweight Small Language Model (SLM)** fine-tuned via **Parameter-Efficient Fine-Tuning (PEFT)** to achieve State-of-the-Art (SOTA) performance on resource-constrained hardware.
- To ensure cross-platform adaptability, we integrate a Retrieval-Augmented Generation (RAG) framework that dynamically ingests database-specific documentation, enabling the model to generalize optimal configuration parameters across heterogeneous Database Management Systems (DBMS).

PROPOSED ARCHITECTURE



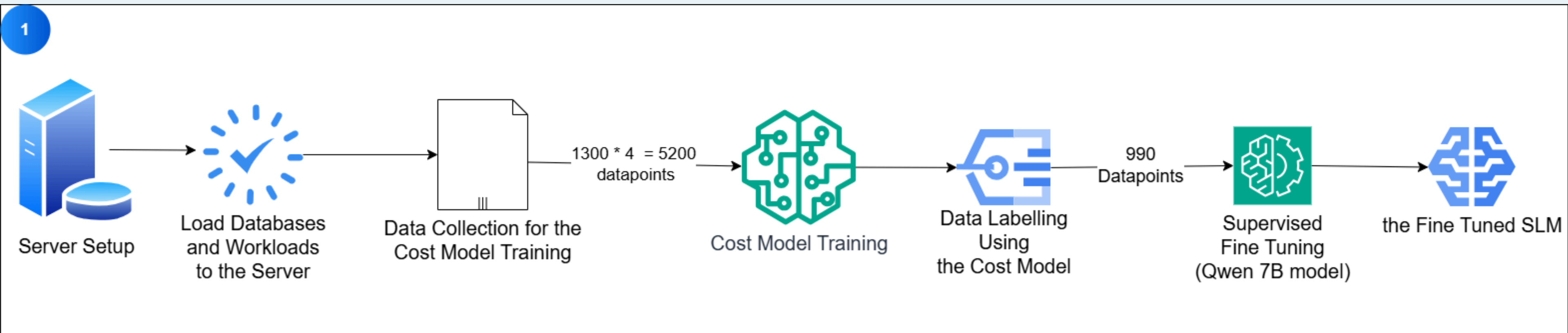
METHODOLOGY

METHODOLOGY



1. Fine Tuning SLM
2. RAG Agent
3. Database Optimization Platform

FINE TUNING SLM



DATA COLLECTION FOR COST MODEL TRAINING

Number of Knobs : 15

Gathered Data Points: 5200

Benchmarks:

- JOB
- SSB
- TPCDS
- TPCH

Number of iterations : 100

Database Server: 8vCPUs, 16GB RAM, 320GB
Storage VM on Digital Ocean

Too Many Knobs to Tune? Towards Faster Database Tuning by Pre-selecting Important Knobs

Konstantinos Kanellis, Ramnathan Alagappan, and Shivaram Venkataraman

University of Wisconsin – Madison

Abstract. To achieve high performance, recent research has shown that it is important to automatically tune the configuration knobs present in database systems. However, as database systems usually have 100s of knobs, auto-tuning frameworks spend a significant amount of time exploring the large configuration space and need to repeat this as workloads change. Given this challenge, we ask a more fundamental question of how many knobs do we need to tune in order to achieve good performance. Surprisingly, we find that with YCSB workload-A on Cassandra, tuning just five knobs can achieve 99% of the performance achieved by the best configuration that is obtained by tuning many knobs. We also show that our results hold across workloads and applies to other systems like PostgreSQL, motivating the need for tools that can automatically filter out the knobs that need to be tuned. Based on our results, we propose an initial design for accelerating auto-tuners and detail some future research directions.

1 Introduction

Tuning configuration parameters is critical for achieving high performance in database systems. This has been true in the past [8, 12, 20] and recent research shows that this is still the case [1, 23, 24]. However, database systems typically have hundreds of configuration knobs that determine the system's runtime behavior [22]; for example, PostgreSQL has about 170 knobs [23] and Apache Cassandra has around 155.

Manually finding the important knobs and their optimal values is thus a daunting task even for experienced practitioners. Consequently, researchers have resorted to automatic ways to tune knobs [9]. Recent auto-tuning methods for database systems typically fall into two classes: search-based approaches [24] and learning-based (ML) methods [1, 23]. In both cases, one starts with a set of knobs and a range of valid values for them, and then the tuning system tries to find the optimal values for each knob to maximize or minimize a specified objective (e.g., throughput or latency). The time it takes to tune is directly dependent on the number of knobs that needs to be tuned [23, 24]. With a larger number of knobs, the search space for search-based methods expands vastly, making it slower to find optimal values for all knobs. Similarly, ML-based approaches require a larger training dataset as we increase the number of knobs, with representative samples required from different values for each knob.

Given this, in this work, we ask the following fundamental question: *how many knobs do we need to tune to achieve good*

performance? Is this a smaller subset of the entire set? To answer this question, we conduct a detailed and systematic measurement-based study. Specifically, we measure the performance of a target system under different configurations and use learning-based techniques to find how many and which knobs need to be tuned for high performance. Our results surprisingly show that tuning just five knobs can achieve 99% of the performance achieved by the best configuration (which tunes many knobs) for the YCSB-A workload in Cassandra.

Motivated by this result, we also extend our experiments across different workloads and systems. Our experiments show that similar results do indeed hold for different workloads (e.g., YCSB-B in Cassandra) and across different systems (PostgreSQL running YCSB-A and YCSB-B). Finally, we also compute the similarity between the sets of important knobs across workloads and find that there is a significant overlap of important knobs across some workloads.

Based on the above results, we argue that it is possible to significantly accelerate auto-tuning approaches by automatically filtering the most important configuration knobs. We propose a new design where we run a filtering or pre-selection phase to determine the important knobs, and we present some initial approaches to realize this design. We also describe future directions for research in understanding how different hardware or metrics change important knobs, and how we can perform auto-tuning while maintaining reliability guarantees.

2 Background and Motivation

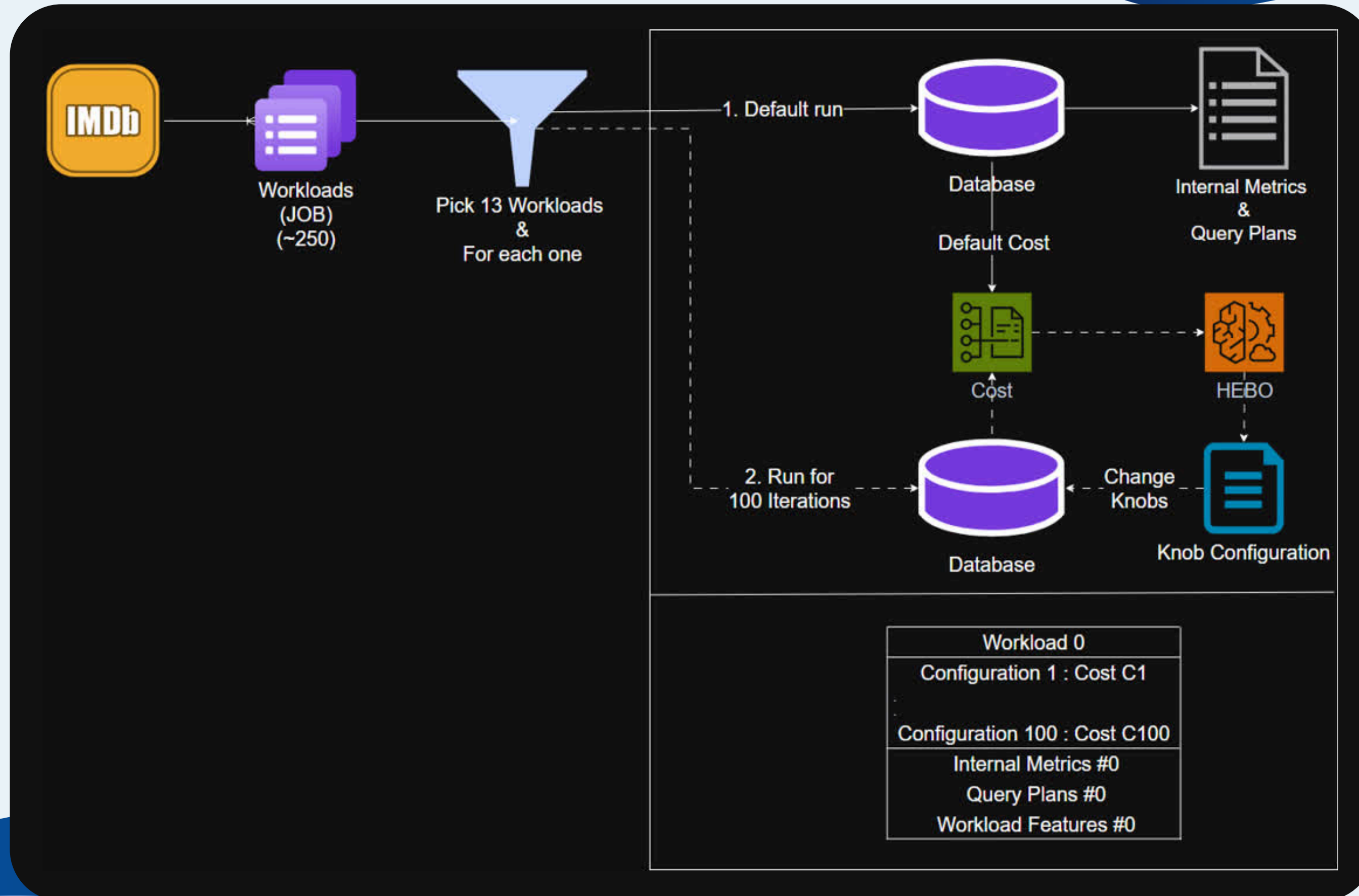
We begin by providing a brief background on auto-tuners and describing the challenges in existing tuning frameworks.

2.1 Automatically Tuning Database Systems

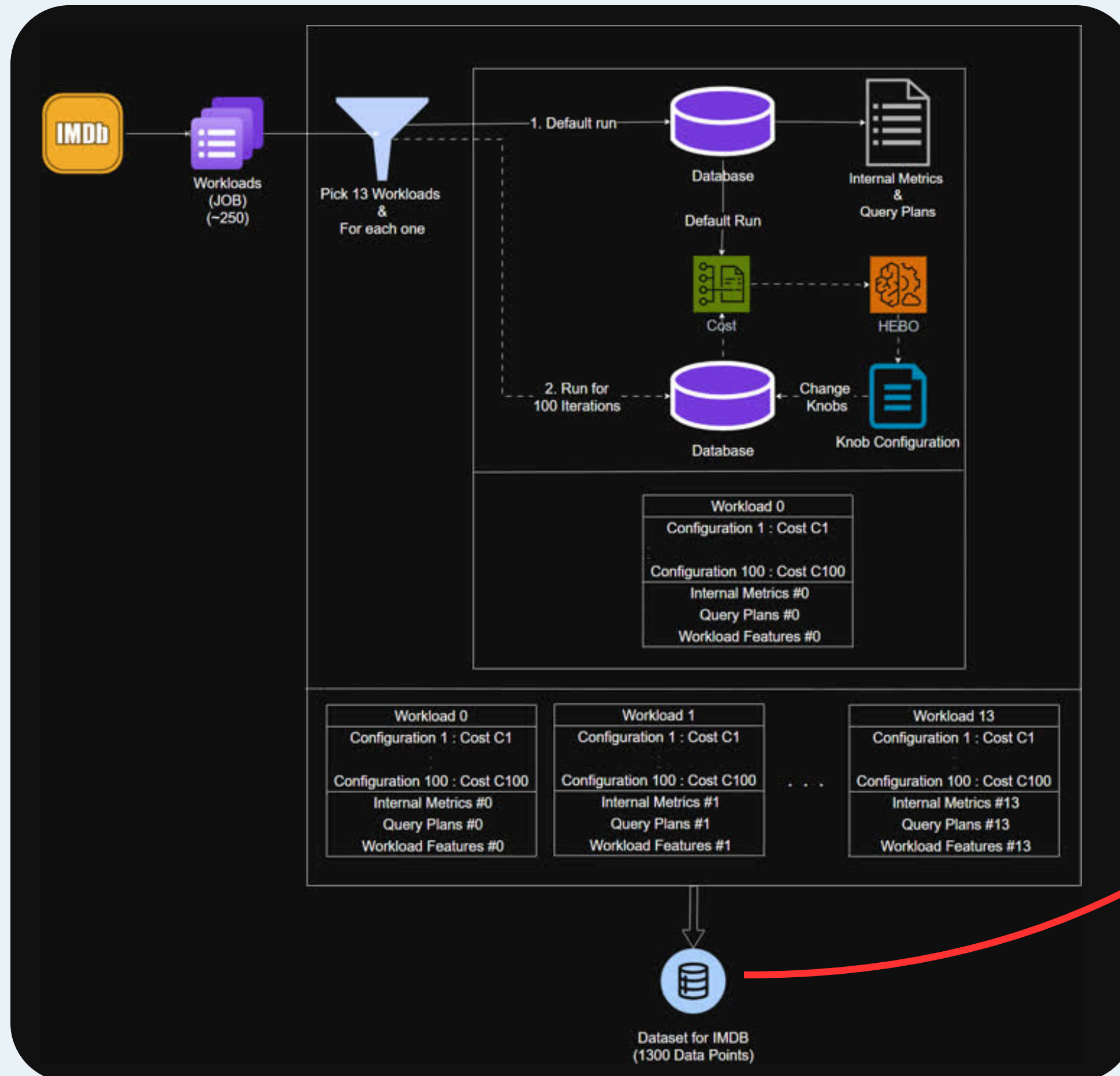
Exploring the entire space of configuration knobs in database systems is time-consuming and intractable. Three factors contribute to this problem. First, modern systems have too many knobs. Second, most knobs take values from a large continuous space, leading to many possible configurations. Finally, it may take several minutes to measure how even a single configuration performs [13, 24].

As a result, recent research has proposed auto-tuning tools for database systems (e.g., Ottertune [1], BestConfig [24], and CDBTune [23]). Most of these tools rely upon an initial offline phase, which either builds a knowledge base that is then used to bootstrap the tuning process for future workloads (by reusing already evaluated configurations), or trains an ML

DATA COLLECTION



DATA COLLECTION



For each
benchmark

TRAINING THE COST MODEL

Dataset used to train the cost model

Input Features

Knob Configurations

Query Plans

Workload Features

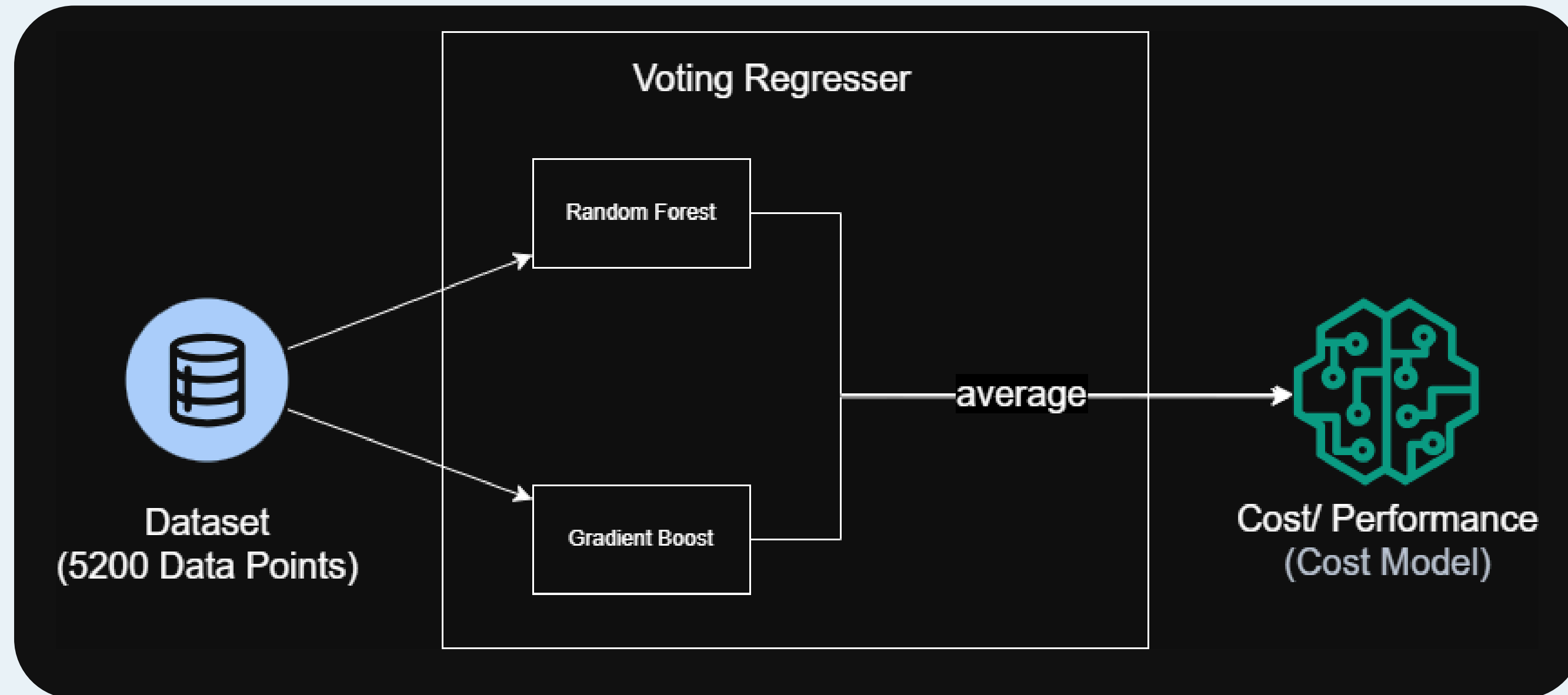
Target

Queries per seconds

Dataset size : 5200

Train/Test Split: 4160/1040

TRAINING THE COST MODEL



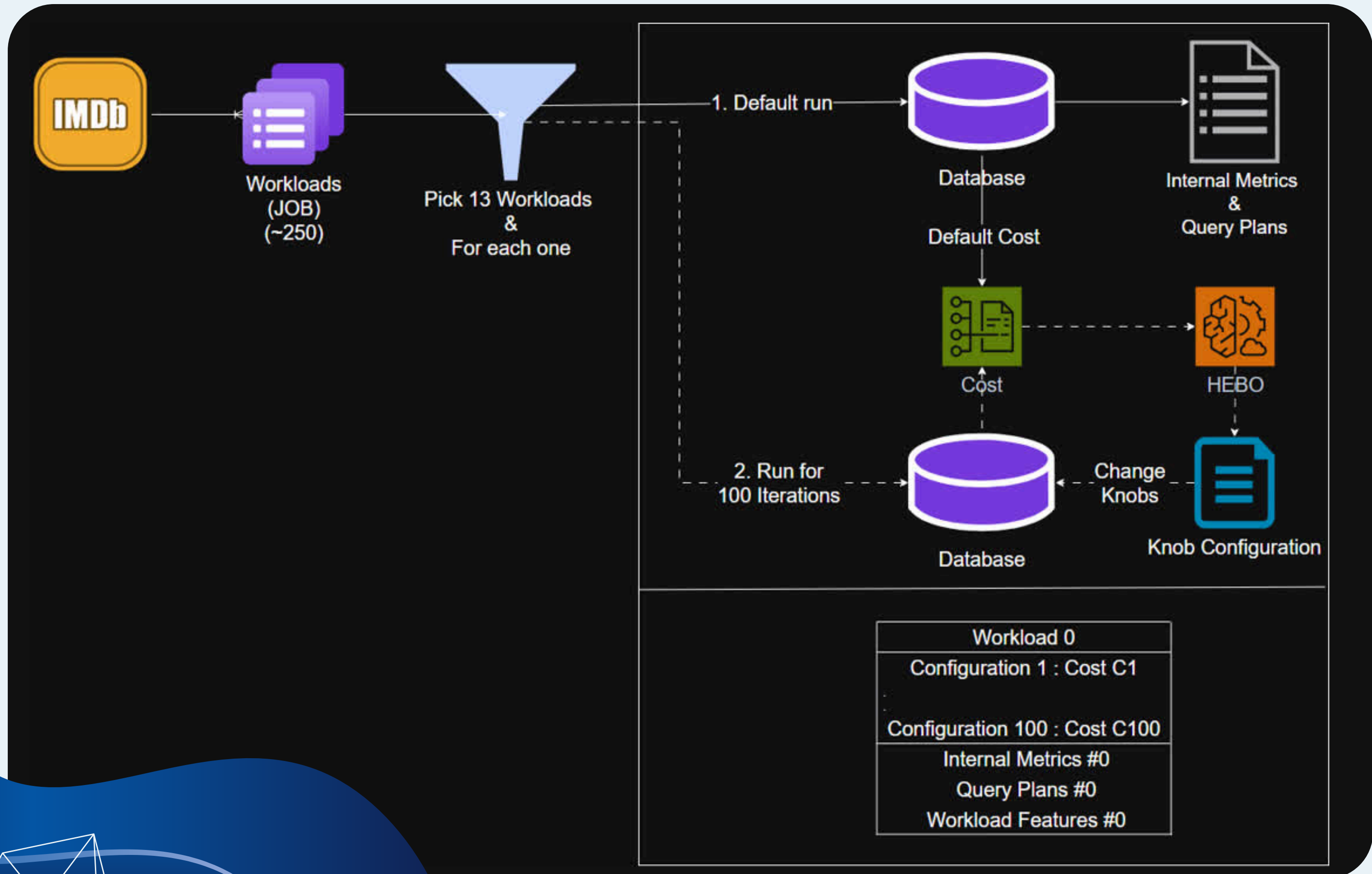
TRAINING THE COST MODEL

Evaluation on the test split

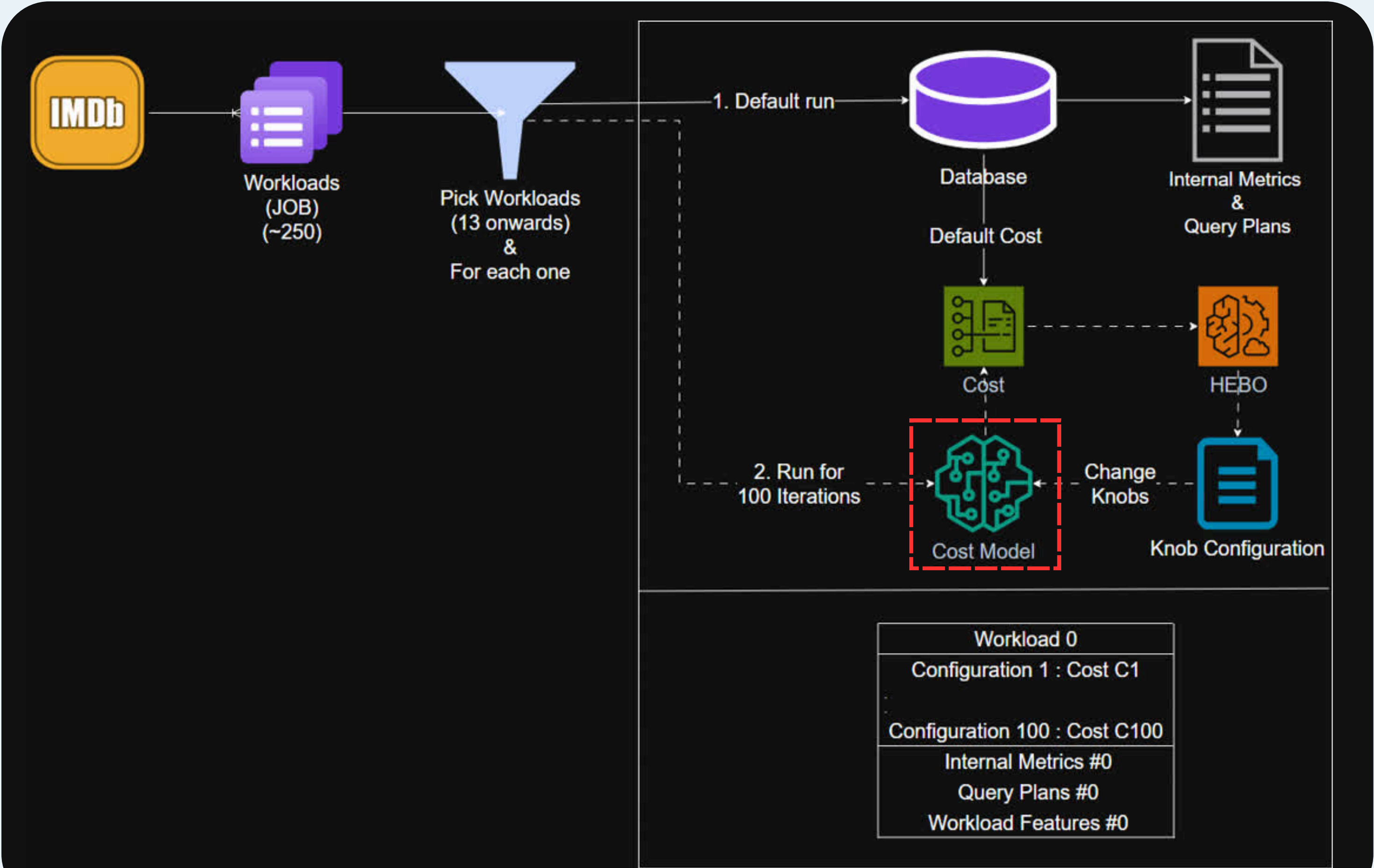
Metric	R2 Score	MAE	RMSE	Spearman's Coefficient
Equation	$1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	$1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$
E2ETune	0.87	n/a	n/a	n/a
Ours	0.8267	0.1849	0.2218	0.8092

- 4.9% R2 Score gap is observed

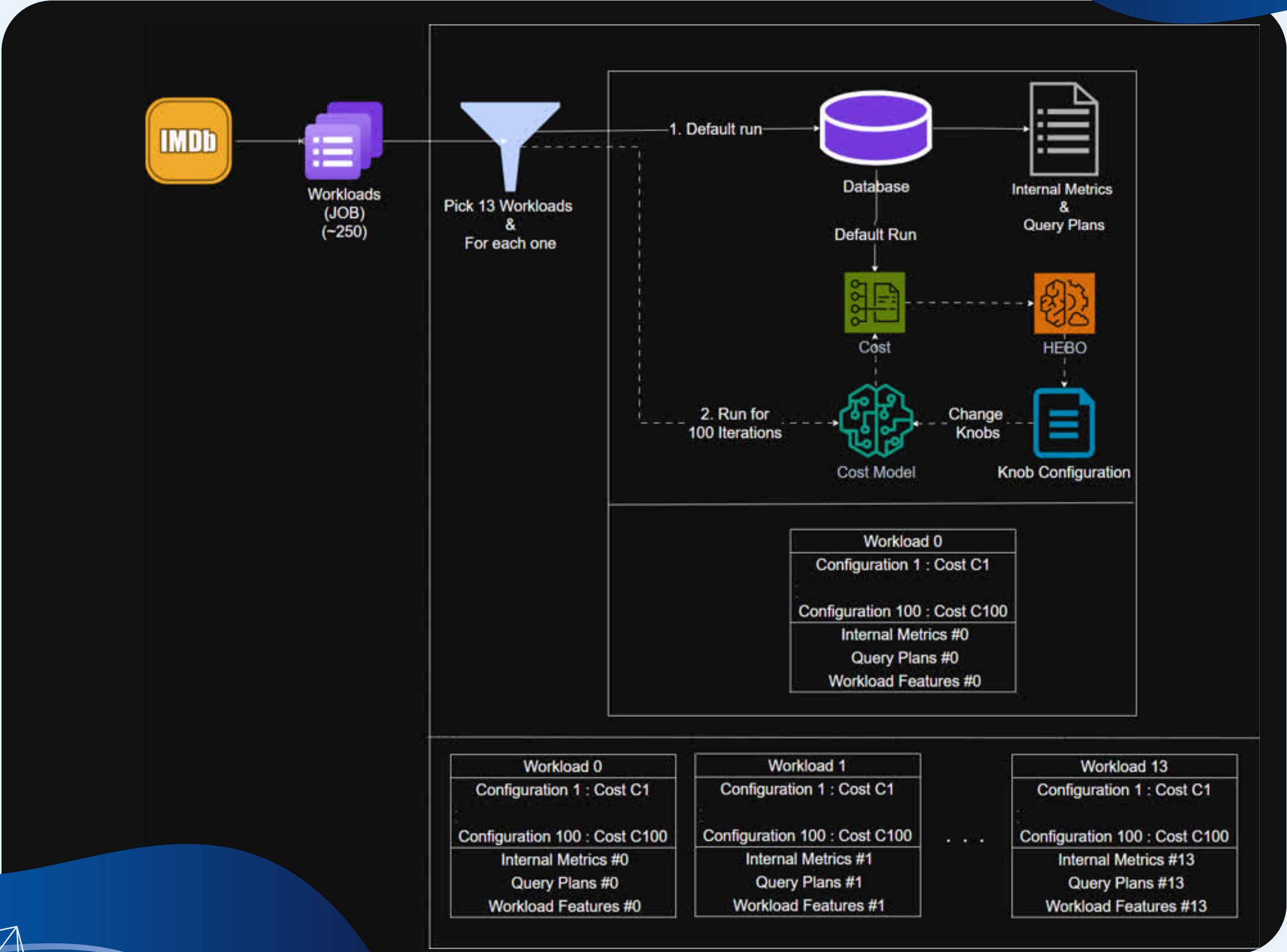
DATA LABELLING



DATA LABELLING



DATA LABELLING



LLM FINE TUNING

Data Transformation

- Input Signals: We aggregate three key data points: **Internal Metrics** (DB state), **Query Plan Features** (execution structure), and **Workload Features** (I/O ratios).
- Discretization Strategy: We experimented with **Direct Continuous Prediction** and **Binned Prediction**.

```
"binned_config": {  
  "shared_buffers": 3,  
  "work_mem": 19,  
  "maintenance_work_mem": 14,  
  "effective_cache_size": 12,  
  "max_connections": 19,  
  "wal_buffers": 4,  
  "checkpoint_completion_target": 6,  
  "checkpoint_timeout": 15,  
  "effective_io_concurrency": 18,  
  "join_collapse_limit": 9,  
  "from_collapse_limit": 4,  
  "bgwriter_delay": 12,  
  "bgwriter_lru_multiplier": 16,  
  "default_statistics_target": 10,  
  "max_parallel_workers_per_gather": 6  
},
```

Binned knob values

```
"raw_config": {  
  "shared_buffers": 605141,  
  "work_mem": 259748,  
  "maintenance_work_mem": 1767954,  
  "effective_cache_size": 2371629,  
  "max_connections": 496,  
  "wal_buffers": 3008,  
  "checkpoint_completion_target": 0.5900715738534927,  
  "checkpoint_timeout": 1533,  
  "effective_io_concurrency": 272,  
  "join_collapse_limit": 18,  
  "from_collapse_limit": 12,  
  "bgwriter_delay": 650,  
  "bgwriter_lru_multiplier": 8.151557922363281,  
  "default_statistics_target": 261,  
  "max_parallel_workers_per_gather": 6  
},
```

Raw knob values

LLM FINE TUNING CONTD

BASE MODEL SELECTION

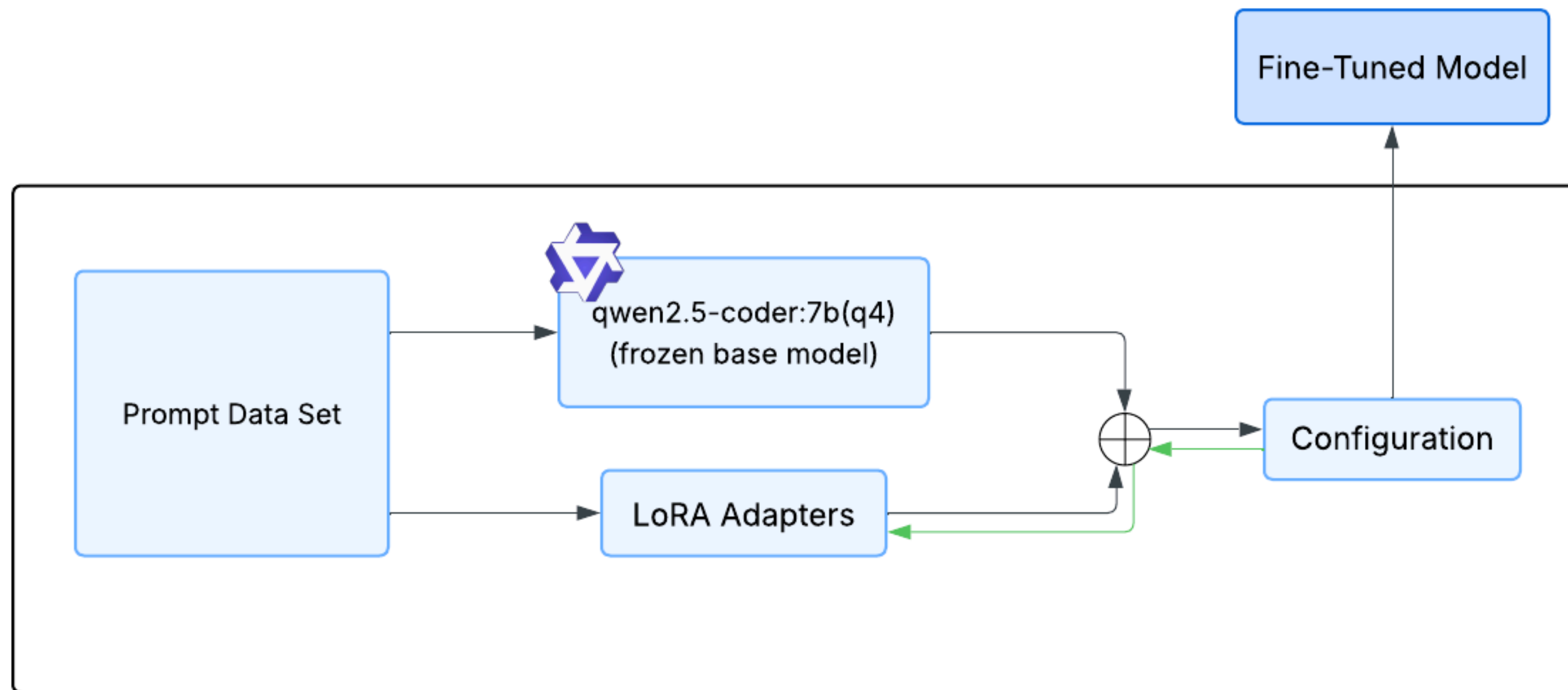
Feature	Qwen 2.5-Coder-7B (Our Choice)	Llama 3.1 8B Instruct (Meta)	Mistral 7B v0.3 (Mistral AI)	DeepSeek-Coder-V2-Lite
Primary Focus	Code & Mathematics	General Chat & Dialogue	General Purpose	Code & Math
Context Window	32k (Effective) / 128k	128k	32k	16k
JSON Adherence	Very High (Strict Syntax)	Moderate (Chatty)	Moderate	High
HumanEval Score	~84-85% (SOTA)	~72-73%	~45-50%	~80%
License	Apache 2.0 (Permissive)	Llama Community (Restrictive)	Apache 2.0	MIT
VRAM (4-bit)	~5.5 GB	~6.0 GB	~5.0 GB	~10 GB (MoE)



LLM FINE TUNING CONTD

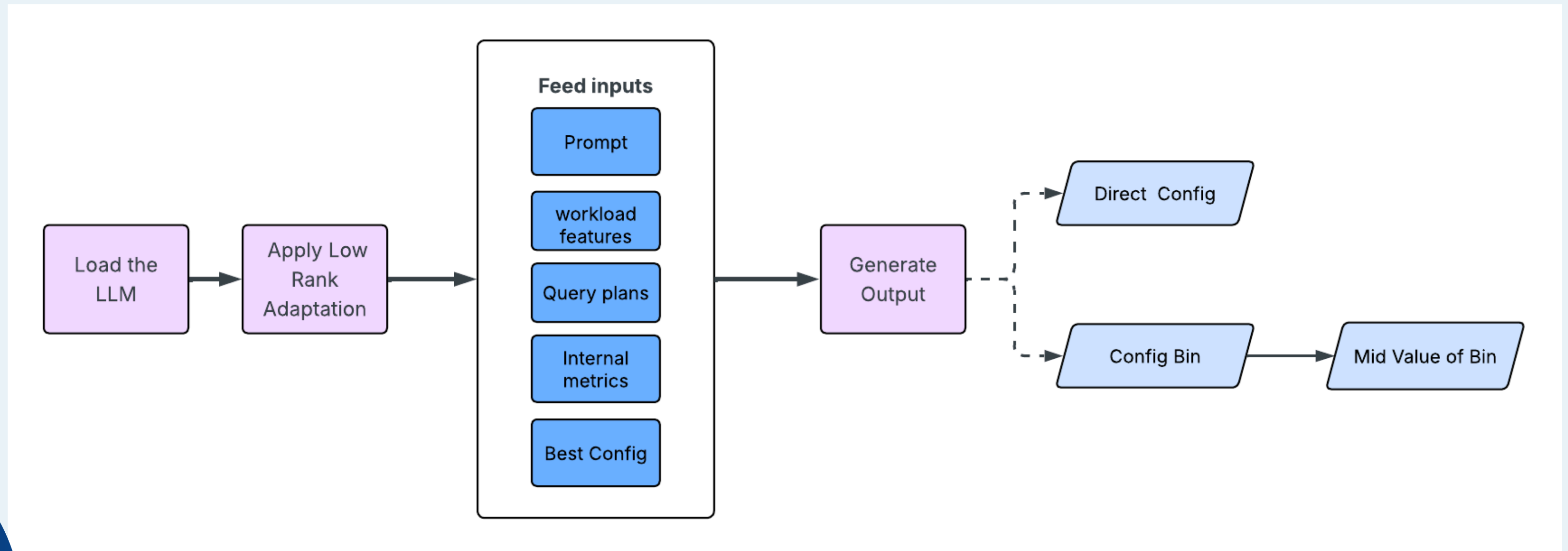
FINE TUNING TECHNIQUE

QLoRA (4-bit Optimization): Freezes base weights and trains lightweight adapters, enabling training on consumer GPUs (T4).



LLM FINE TUNING

NVIDIA RTX 6000 ADA
48GB VRAM



LLM SFT RESULTS

Model	with Direct Config Output	with Binned Output
Train_loss	0.4131	0.2562
Validation_loss	0.4313	0.2732
Test_loss	0.4293	0.2788

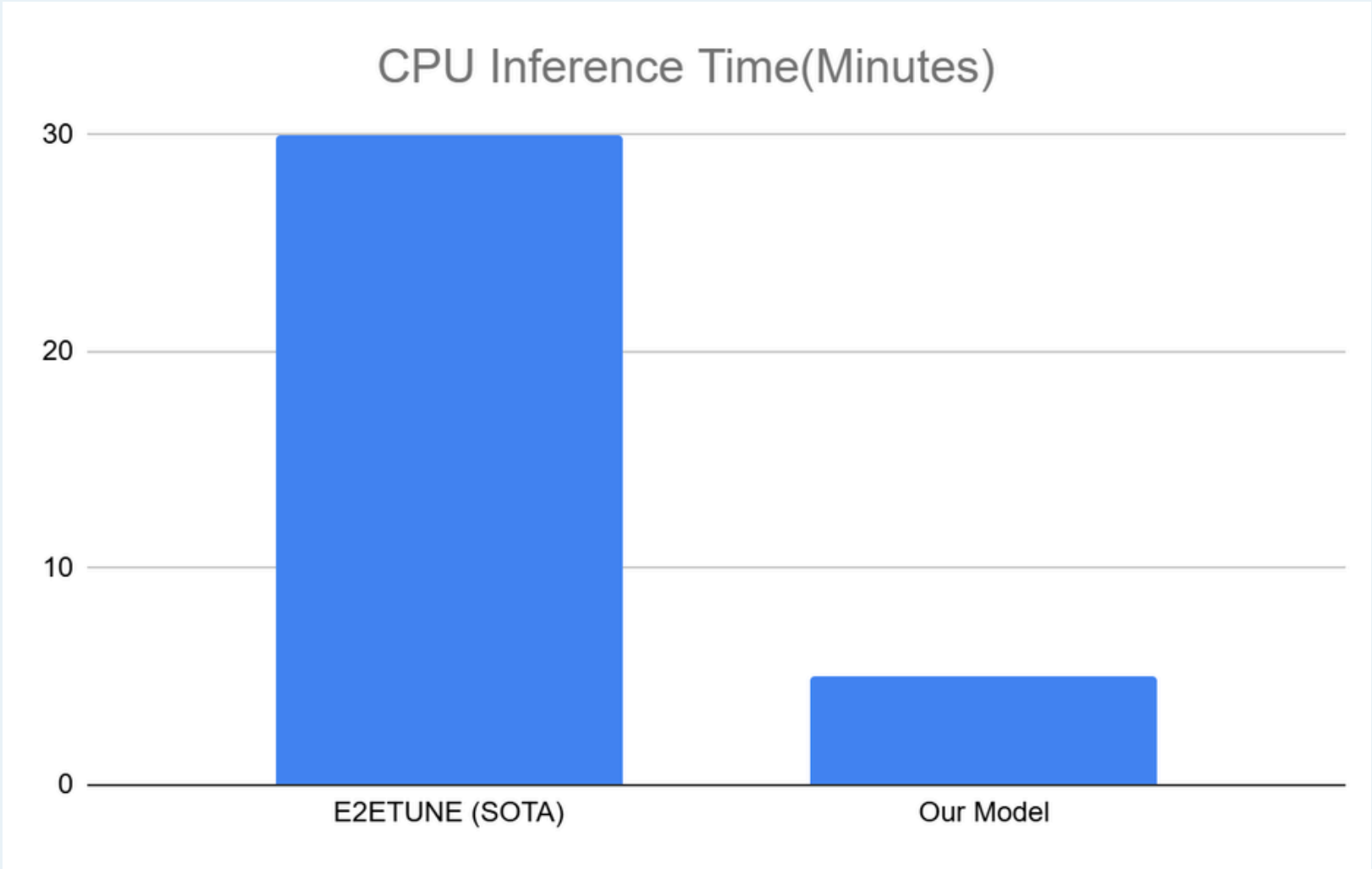
The Binning approach outperformed Direct Prediction, as LLMs struggle with precise floating-point arithmetic but excel at categorical selection.

EVALUATION RESULTS

1. Average CPU Inference Time (over 5 inferences)

E2ETUNE (SOTA)	Our Model
30 mins	5 mins

- 83.33% Improvement in CPU inference time



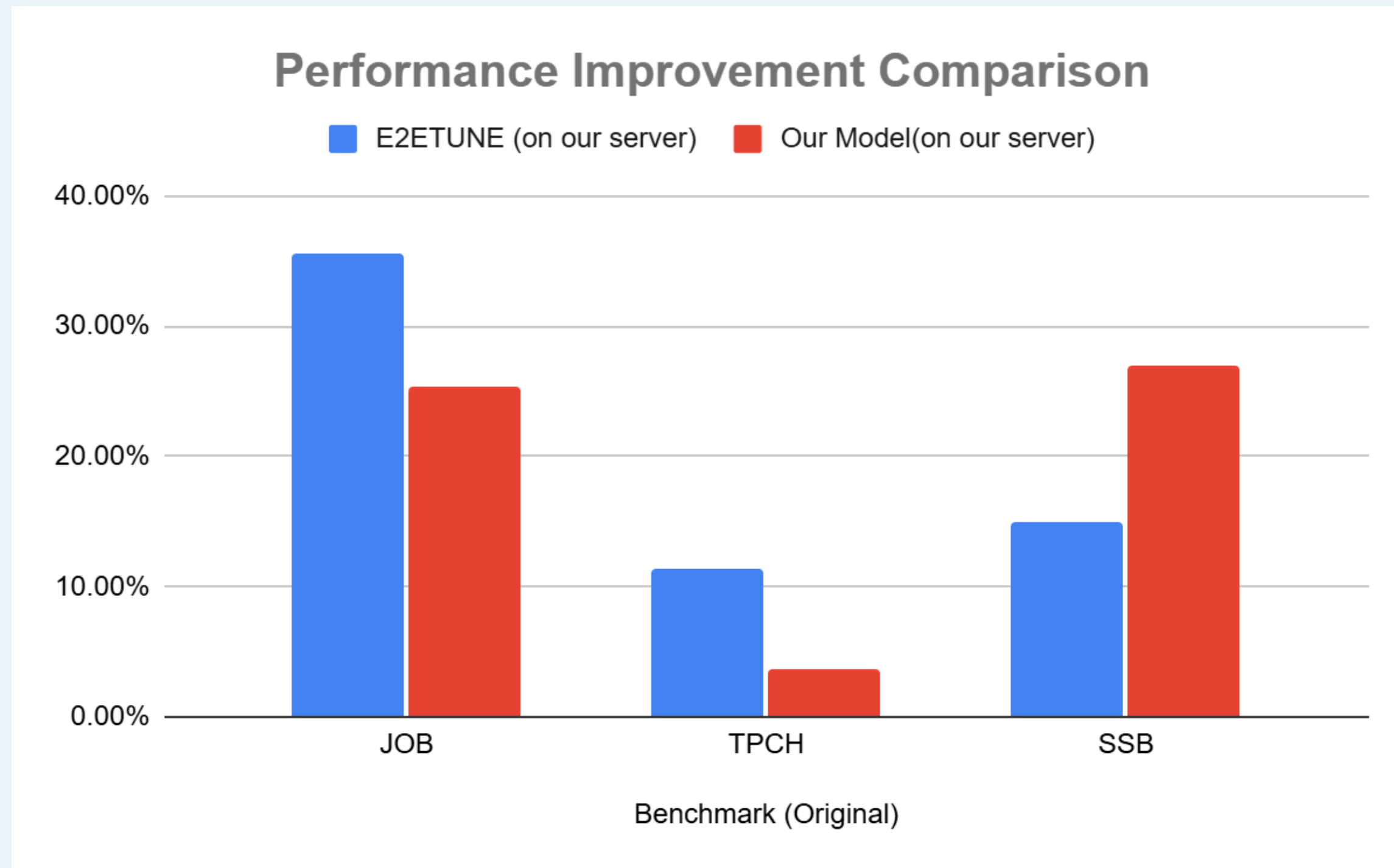
EVALUATION RESULTS

2. Performance Improvement

$$\frac{OptimizedTPS - DefaultTPS}{DefaultTPS} \times 100$$

Benchmark (Original)	E2ETUNE (reported results)	E2ETUNE (on our server)	Our Model(on our server)
JOB	58.00%	35.60%	25.34%
TPCH	71.00%	11.41%	3.65%
SSB	47.00%	15.01%	26.96%
Average	58.6%	20.6%	18.65%

CONTINUED..



- E2ETUNE shows stronger performance on JOB and TPCH benchmarks.
- Our model performs comparatively better on SSB.
- Overall, our model attains a competitive average improvement of 18.65%, closely approaching E2ETUNE's 20.60%.

REASONS FOR THE GAP

EXPERIMENTAL SETTING	E2ETUNE (SOTA)	OUR MODEL
Training dataset size	2953	990
Number of Benchmarks	10	4
Benchmarks Used	TPC-H, TPC-DS, JOB, SSB, SSBflat, TPC-C, Twitter, Wikipedia, Smallbank, and YCSB	TPC-H, TPC-DS, JOB, SSB
Database Engine	PostgreSQL 12.2	PostgreSQL 12.2
Knobs	45	15
CPU	Intel Xeon E5 2650 v4(12cores, 24threads)	AMD EPYC (8 Cores, 8 Threads Virtualized)
RAM	64GB	16GB

- Initial datasets were captured on shared infrastructure, introducing environmental noise.
- A comparatively smaller dataset has been used. (Only 990 workloads.)

DEMO VIDEO

Visual Studio Code interface showing a terminal session and file explorer.

Terminal: (llm_venv) root@ai4db-larger:/home/E2ETune-AI4DB/inference_test#

File Explorer (HOME [SSH: AI4DB-LARGER]):

- E2ETune-AI4DB
 - inference_test
 - default_runner.py
 - e2etune-inference.ipynb
 - genknob-tuner.ipynb
 - get_model_output.py
 - job_0-3_first_3.sql
 - job_12.sql
 - job-3and12.sql
 - job.sql
 - run_model_output.py
 - runCM_8_candidates.py
 - runDB_8_candidates.py
 - runDB_8_E2E_candidates.py
 - sft_llm_test.ipynb
 - ssb.sql
 - SUBFOLDER_STRUCTURE.md
 - test_data.csv
 - test.json
 - tpch.sql
 - internal_metrics

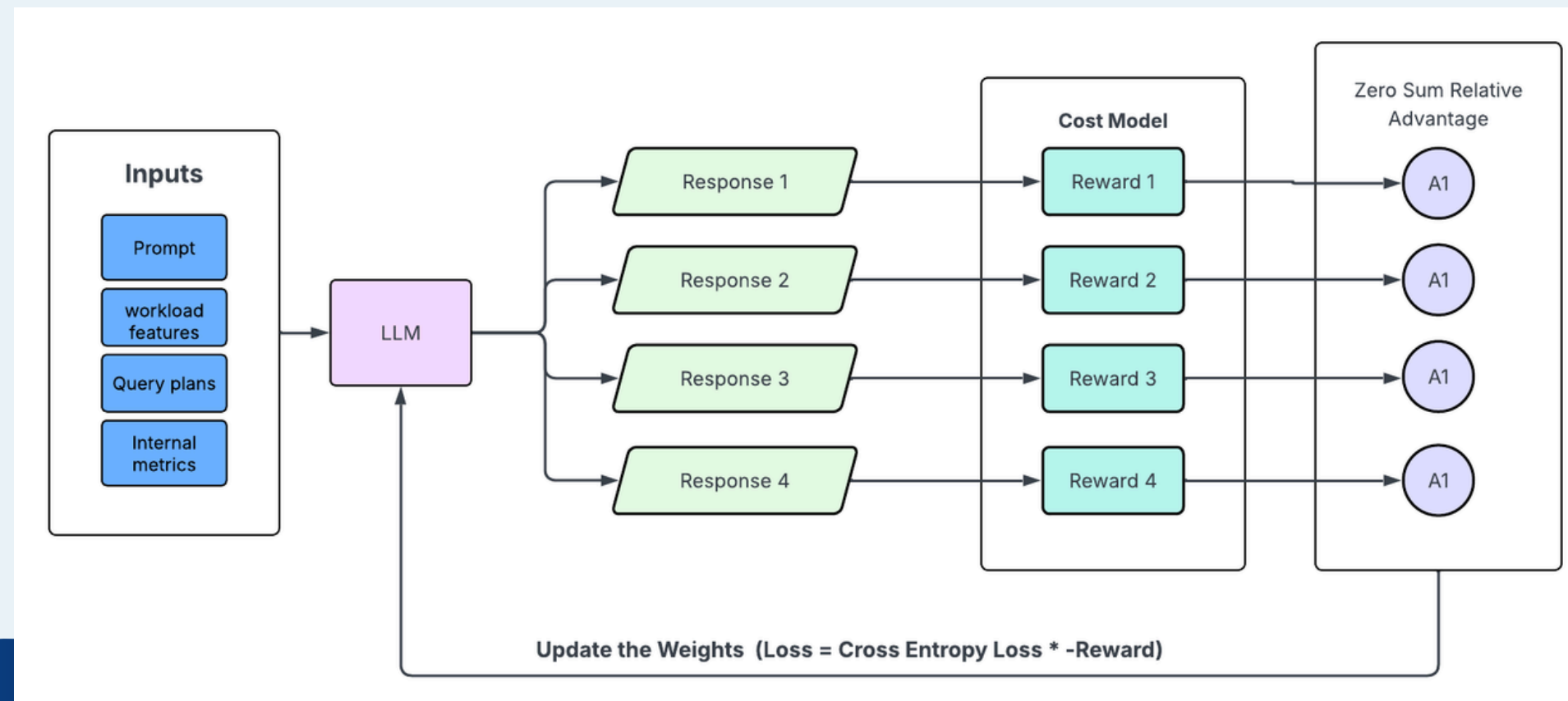
Bottom Bar: SSH: ai4db-larger, master*, 0 errors, 0 warnings, 1 info, Ln 1, Col 1, Spaces: 4, UTF-8, LF, Python, 3.10.12 (.venv)

SUGGESTED IMPROVEMENTS

1. IMPROVE COST MODEL GENERALIZATION

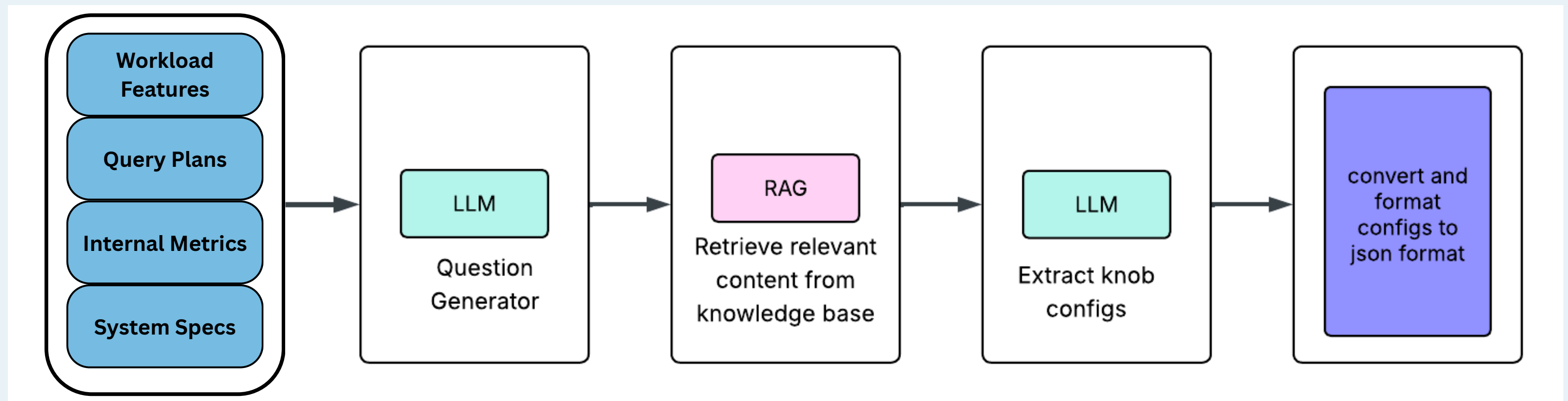
- Implement K-Fold Cross-Validation (3-fold / 5-fold) to rigorously test the cost model's stability across unseen workloads.
- Experiment with other model types beyond regression, such as Neural Networks.
- Generalize the cost model for different server specs.

2. INTEGRATION OF REINFORCEMENT LEARNING (ONLINE FINETUNING WITH GRPO)



RAG AGENT

METHODOLOGY



SAMPLE OUTPUT

```
STRUCTURED KNOB RECOMMENDATIONS (System RAM: 40 GB)
```

```
1. work_mem
```

```
Original Recommendation: 10 - 50 MB
```

```
Converted Value: 10240 - 51200
```

```
Range: [10240 - 51200]
```

```
Target Unit: KB
```

```
Workload Context: OLAP
```

```
Reasoning: Allows most sorts and hashes to stay in memory, reducing disk spills for analytical queries.
```

```
Schema: type=integer, range=[64, 2147483647], default=4096
```

```
2. maintenance_work_mem
```

```
Original Recommendation: 5 % of total system RAM
```

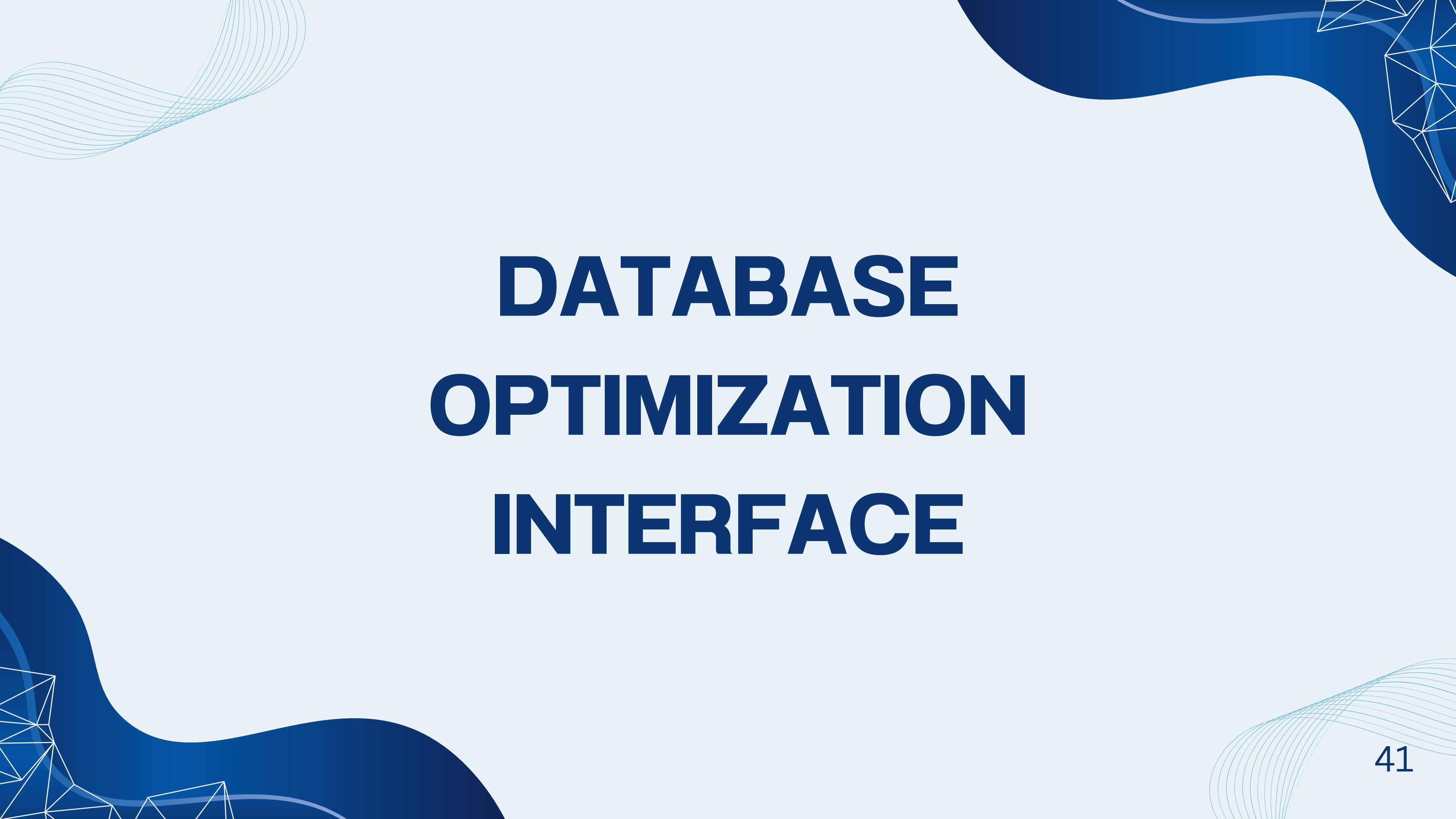
```
Converted Value: 2097152
```

```
Target Unit: KB
```

```
Workload Context: OLAP
```

```
Reasoning: Provides enough memory for maintenance tasks like index builds and vacuum without starving other processes.
```

```
Schema: type=integer, range=[1024, 2147483647], default=65536
```



DATABASE OPTIMIZATION INTERFACE

USER INTERFACE

FileObjectToolsHelp

BROWSER

▼ Servers (0)

No servers registered

DashboardOptimized Configuration

▶ Optimize

Your Database

Save a new database connection

Connection Name

My Production Database

Credentials

Connection String

Host

localhost

Port

5432

Database

mydatabase

Username

postgres

Password

.....

+ Save Connection

📁 Saved Databases (0)

Workload

Upload a workload file or paste your SQL queries

📄 Paste

📁 Upload

Paste your SQL workload here...

System Specifications

Configure your system hardware details

CPU Cores

e.g., 8

RAM (GB)

e.g., 16

Hard Drive Type

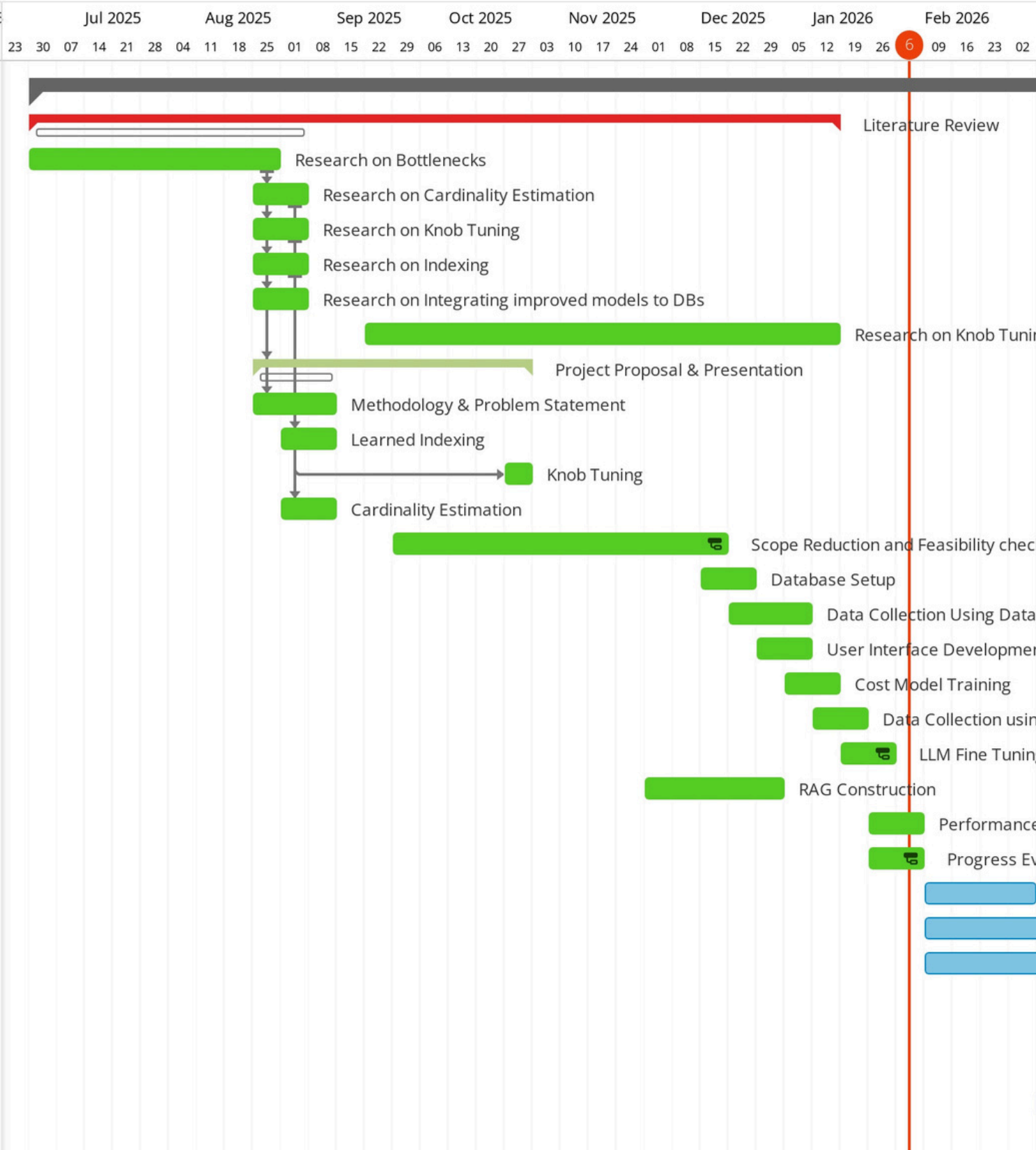
SSD (Solid State Drive) ▼

TIMELINE & WORK DISTRIBUTION

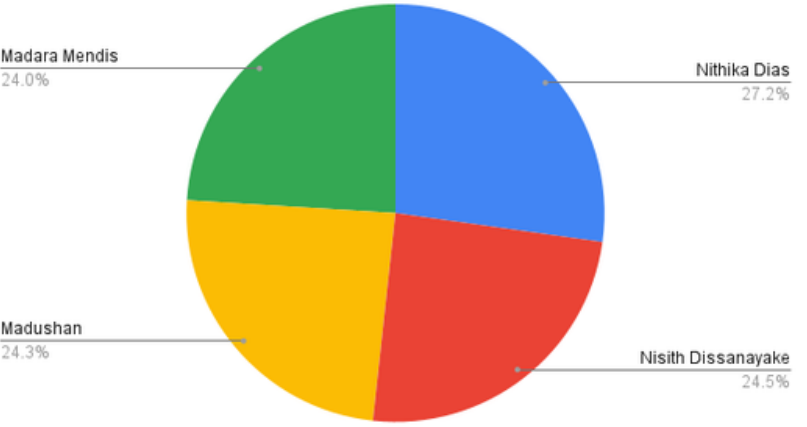
AI4DB

Read-only view, generated on 06 Feb 2026

	ACTIVITIES	ASSIGNEE	EH	START	DUE	STATUS	%
	AI4DB:		2030h	04/Jul	10/May		58%
	✔ Literature Review	MM, ND, MS, ND	660h	04/Jul	14/Jan	Finished	100%
2	✔ Research on Bottlenecks	ND, MS, ND, MM	220h	04/Jul	25/Aug	Finished	100%
3	✔ Research on Cardinality...	Nisith Dissanay...	70h	25/Aug	05/Sep	Finished	100%
4	✔ Research on Knob Tuning	Madara Mendis	70h	25/Aug	05/Sep	Finished	100%
5	✔ Research on Indexing	Madushan Soor...	50h	25/Aug	05/Sep	Finished	100%
6	✔ Research on Integrating...	Nithika Dias	50h	25/Aug	05/Sep	Finished	100%
7	✔ Research on Knob Tuning	MM, ND, ND, MS	200h	25/Sep	14/Jan	Started	100%
	✔ Project Proposal & Presenta...	MM, MS, ND, ND	120h	26/Aug	30/Oct	Started	100%
9	✔ Methodology & Proble...	Nithika Dias	40h	26/Aug	10/Sep	Started	100%
10	✔ Learned Indexing	Madushan Soor...	40h	05/Sep	10/Sep	Finished	100%
11	✔ Knob Tuning	Madara Mendis	20h	27/Oct	30/Oct	Finished	100%
12	✔ Cardinality Estimation	Nisith Dissanay...	20h	05/Sep	10/Sep	Finished	100%
	✔ Scope Reduction and Feasib...	MM, MS, ND, ND	40h	29/Sep	17/Dec	Finished	100%
16	✔ Database Setup	ND, MS	10h	21/Dec	23/Dec	Finished	100%
17	✔ Data Collection Using Data...	Nithika Dias	60h	23/Dec	09/Jan	Finished	100%
18	✔ User Interface Development	Madushan Soor...	20h	02/Jan	07/Jan	Finished	100%
19	✔ Cost Model Training	Nisith Dissanay...	20h	10/Jan	17/Jan	Finished	100%
20	✔ Data Collection using Cost ...	Nithika Dias	30h	17/Jan	20/Jan	Finished	100%
	✔ LLM Fine Tuning	MM, ND	30h	21/Jan	26/Jan	Finished	100%
24	✔ RAG Construction	Madara Mendis	20h	01/Dec	31/Dec	Finished	100%
25	✔ Performance Evaluation	ND, ND	20h	27/Jan	02/Feb	Finished	100%
	✔ Progress Evaluation and Pr...	MM, MS, ND, ND	20h	31/Jan	06/Feb	Started	100%
29	✔ Backend Development	Madushan Soor...	20h	09/Feb	03/Mar	Not started	0%
30	✔ LLM Fine Tuning with GRPO	MM, ND	40h	10/Feb	10/Mar	Not started	0%
31	✔ Data Collection for Differen...	MS, ND	50h	15/Feb	20/Mar	Not started	0%
32	✔ Cost Model Fine Tuning for ...	MM, ND	20h	21/Mar	26/Mar	Not started	0%
33	✔ Integration Testing of RAG a...	MM, MS, ND, ND	220h	27/Mar	10/Apr	Not started	0%
34	✔ System Test	MM, MS, ND, ND	325h	17/Apr	28/Apr	Not started	0%
35	✔ Final Performance Evaluation	MM, MS, ND, ND	220h	12/Mar	30/Apr	Not started	0%
36	✔ Final Report & Research Pa...	MS, MM, ND, ND	85h	01/May	10/May	Not started	0%



Total Hours (2030 hrs)



CONCLUSION

- 58% of the project work has been completed according to the planned timeline.
- Due to infrastructure limitations, we could not fully reproduce the performance reported in Huang et al., 2025 (E2ETune).
- Our model achieves an 83.33% reduction in CPU inference time compared to E2ETune.
- On our server, the model shows a 9.4% performance improvement gap relative to E2ETune.
- Next steps: Mitigate the performance gap through online fine-tuning and cost model improvements.

REFERENCES

- [1] Immanuel Trummer. 2022. DB-BERT: a Database Tuning Tool that" Reads the Manual". In Proceedings of the 2022 International Conference on Management of Data. 190–203.
- [2] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2023. GPTuner: A Manual Reading DatabaseTuningSystemviaGPT-GuidedBayesianOptimization. arXiv preprint arXiv:2311.03157 (2023).
- [3] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, et al. 2019. An end-to-end automatic cloud database tuning system using deep reinforcement learning. In Proceedings of the 2019 International Conference on Management of Data. 415–432.
- [4] X. Huang, H. Li, J. Zhang, X. Zhao, Z. Yao, Y. Li, T. Zhang, J. Chen, H. Chen, and C. Li, “E2ETune: End-to-End Knob Tuning via Fine-Tuned Generative Language Model,” arXiv preprint arXiv:2404.11581, Mar. 2025. doi: 10.48550/arXiv.2404.11581.
- [5] Alexander I Cowen-Rivers, Wenlong Lyu, Zhi Wang, Rasul Tutunov, Hao Jianye, Jun Wang, and Haitham Bou Ammar. 2020. Hebo: Heteroscedastic evolutionary bayesian optimisation. arXiv preprint arXiv:2012.03826 (2020), 7.
- [6] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In Learning and Intelligent Optimization: 5th International Conference, LION 5, Rome, Italy, January 17-21, 2011. Selected Papers 5. Springer, 507–523.
- [7] Xinyi Zhang, Hong Wu, Yang Li, Zhengju Tang, Jian Tan, Feifei Li, and Bin Cui. 2023. An Efficient Transfer Learning Based Configuration Adviser for Database Tuning. Proceedings of the VLDB Endowment 17, 3 (2023), 539–552.
- [8] <https://pgtune.leopard.in.ua>.
- [9] J. Ansel, S. Kamil, K. Veeramachaneni, and et al. Opentuner: An extensible framework for program autotuning. In Parallel architectures and compilation, 2014.
- [10] Y. Zhu and et al. Bestconfig: tapping the performance potential of systems via automatic configuration tuning. In SoCCer, 2017.



THANK YOU